

CYBER DEFENSE ANALYST HANDBOOK

Expanded Ebook — Volume 2

15 new defensive cybersecurity modules. This volume intentionally avoids repeating the Windows threat-hunting, Sysinternals, memory forensics, kernel/rootkit, UEFI, Android, reverse-engineering, malware-lab, YARA/Sigma, EDR/SIEM, hardening and playbook material from the earlier ebook.

Chapter	New topic
01	Security Architecture & Zero-Trust Design
02	Identity & Access Security
03	Active Directory Security & Attack-Surface Review
04	Email Security & Phishing Investigation
05	Web Application Security Fundamentals
06	API Security & Service-to-Service Trust
07	Cloud Security Monitoring
08	Container & Kubernetes Defense
09	Vulnerability Management & Patch Strategy
10	Data Security, DLP & Information Classification
11	Security Logging & Detection Engineering
12	Threat Modeling for New Projects
13	Third-Party & Supply-Chain Security
14	Business Continuity, Backups & Recovery Engineering
15	Security Governance, Metrics & Analyst Career Skills

Chapter 01: Security Architecture & Zero-Trust Design

This chapter is written as a practical defensive reference. Apply exercises only to systems, identities, applications, and environments for which you have explicit authorization.

Why architecture matters

Security incidents become easier to contain when trust boundaries are explicit. Instead of assuming that a user, device, network segment, or application is trustworthy because it is inside the corporate network, design access around identity, device posture, application context, and the minimum privileges required.

Core principles

Separate identity, workload, data, management, and user-access planes. Use strong authentication, short-lived access where practical, segmentation, policy enforcement at multiple layers, and continuous verification. A mature architecture assumes that one control can fail.

Design exercise

Draw a simple organization with users, SaaS applications, internal applications, administrators, endpoints, servers, and sensitive data. Mark every trust boundary. For each boundary ask what authenticates the actor, what authorizes the action, what is logged, and how access is revoked.

Analyst checklist

■ Identify trust boundaries ■ Map privileged paths ■ Identify single points of failure ■ Verify logging at enforcement points ■ Review emergency-access procedures

Chapter takeaway

The objective is not to memorize individual tools or alerts. Build repeatable reasoning: establish context, define the hypothesis, collect independent evidence, assess business impact, choose the least disruptive effective control, and document what remains uncertain.

Chapter 02: Identity & Access Security

This chapter is written as a practical defensive reference. Apply exercises only to systems, identities, applications, and environments for which you have explicit authorization.

Identity as the control plane

Modern environments often make identity the common layer connecting SaaS, cloud consoles, internal applications, remote access, and administration. Protecting identity therefore requires more than a strong password: authentication strength, session controls, privilege design, recovery processes, and monitoring all matter.

Privilege management

Separate ordinary accounts from administrative identities. Use role-based access, time-limited elevation where available, approval for sensitive operations, and periodic access reviews. Service identities should have narrow permissions and clear ownership.

Detection ideas

Useful signals include unusual sign-in geography, impossible travel patterns, repeated authentication failures, new authentication methods, unexpected privilege changes, dormant-account use, and access to resources outside a user's normal role. None of these is automatically malicious; context determines severity.

Investigation workflow

Start with identity → device → application → action → time. Then compare the event with the user's role, normal working hours, historical behavior, and recent administrative changes.

Chapter takeaway

The objective is not to memorize individual tools or alerts. Build repeatable reasoning: establish context, define the hypothesis, collect independent evidence, assess business impact, choose the least disruptive effective control, and document what remains uncertain.

Chapter 03: Active Directory Security & Attack-Surface Review

This chapter is written as a practical defensive reference. Apply exercises only to systems, identities, applications, and environments for which you have explicit authorization.

Directory risk model

A directory is a map of identities, groups, computers, policies, and relationships. Security teams should understand how privilege can flow through those relationships without attempting to exploit them.

High-value review areas

Review highly privileged groups, nested membership, service accounts, stale computer accounts, delegation settings, legacy authentication dependencies, administrative workstations, and policy inheritance.

Exposure reduction

Reduce standing privilege, separate administrative tiers, remove stale identities, protect privileged accounts with stronger controls, and monitor sensitive directory changes. Document business exceptions rather than leaving them unexplained.

Audit exercise

Create a quarterly directory review containing privileged groups, owners, last-review dates, exceptions, and remediation status. The goal is not a perfect snapshot; it is repeatable governance.

Chapter takeaway

The objective is not to memorize individual tools or alerts. Build repeatable reasoning: establish context, define the hypothesis, collect independent evidence, assess business impact, choose the least disruptive effective control, and document what remains uncertain.

Chapter 04: Email Security & Phishing Investigation

This chapter is written as a practical defensive reference. Apply exercises only to systems, identities, applications, and environments for which you have explicit authorization.

Phishing analysis

Email investigations should distinguish delivery, sender authenticity, user interaction, payload behavior, and downstream account activity. A suspicious message can be malicious even when it contains no attachment; links, impersonation, QR codes, and social engineering can be equally important.

Header and message analysis

Record sender and recipient, timestamps, authentication results, reply-to behavior, URLs, attachment metadata, and message-routing information. Preserve the original message when possible rather than relying only on screenshots.

User-impact assessment

Determine whether the recipient opened a link, submitted credentials, downloaded a file, replied, or forwarded the message. Then search for the same campaign across other recipients.

Response

Remove or quarantine related messages, reset or protect affected identities when justified, block confirmed malicious infrastructure, and communicate clearly with users. Document the evidence that led to each action.

Chapter takeaway

The objective is not to memorize individual tools or alerts. Build repeatable reasoning: establish context, define the hypothesis, collect independent evidence, assess business impact, choose the least disruptive effective control, and document what remains uncertain.

Chapter 05: Web Application Security Fundamentals

This chapter is written as a practical defensive reference. Apply exercises only to systems, identities, applications, and environments for which you have explicit authorization.

Application attack surface

A web application exposes interfaces, authentication flows, APIs, business logic, file handling, third-party components, and administrative functions. Defensive testing should focus on reducing unintended behavior and protecting data.

Secure design questions

For every sensitive operation ask: who can perform it, what proves identity, what authorizes it, what data can it access, how input is validated, how errors are handled, and what security event is logged.

Common weakness classes

Important defensive categories include broken access control, insecure authentication, injection flaws, unsafe file handling, insecure deserialization, weak security configuration, and dependency vulnerabilities.

Review method

Use threat modeling, code review, automated scanning, dependency review, and authorized testing together. Findings should be prioritized by exploitability, exposure, business impact, and availability of compensating controls.

Chapter takeaway

The objective is not to memorize individual tools or alerts. Build repeatable reasoning: establish context, define the hypothesis, collect independent evidence, assess business impact, choose the least disruptive effective control, and document what remains uncertain.

Chapter 06: API Security & Service-to-Service Trust

This chapter is written as a practical defensive reference. Apply exercises only to systems, identities, applications, and environments for which you have explicit authorization.

Why APIs need separate attention

APIs frequently expose more functionality than a user interface. They also create machine-to-machine trust relationships that can become difficult to inventory.

Key controls

Use strong service authentication, explicit authorization, schema validation, rate controls, secrets management, API inventory, version management, and meaningful audit logs. Avoid treating an internal network location as sufficient authorization.

Monitoring

Track unusual request volume, unexpected endpoints, new clients, abnormal error rates, privilege-sensitive operations, and changes in API credentials. Correlate requests with identities and workloads.

Governance

Every production API should have an owner, data classification, authentication method, authorization model, logging expectations, dependency list, and retirement process.

Chapter takeaway

The objective is not to memorize individual tools or alerts. Build repeatable reasoning: establish context, define the hypothesis, collect independent evidence, assess business impact, choose the least disruptive effective control, and document what remains uncertain.

Chapter 07: Cloud Security Monitoring

This chapter is written as a practical defensive reference. Apply exercises only to systems, identities, applications, and environments for which you have explicit authorization.

Cloud visibility

Cloud investigations depend on control-plane events, identity logs, workload telemetry, configuration history, and data-access records. A single server-centric monitoring model is insufficient.

What to monitor

Prioritize authentication, privilege changes, creation or deletion of resources, security-policy changes, key management activity, public exposure changes, and access to sensitive data.

Baseline

Establish expected administrative roles, regions, resource types, deployment pipelines, and automation identities. Alert on meaningful deviations rather than every unusual event.

Incident workflow

Identify account → affected cloud service → changed resource → data or workload impact → persistence or continued access → containment → configuration restoration → validation.

Chapter takeaway

The objective is not to memorize individual tools or alerts. Build repeatable reasoning: establish context, define the hypothesis, collect independent evidence, assess business impact, choose the least disruptive effective control, and document what remains uncertain.

Chapter 08: Container & Kubernetes Defense

This chapter is written as a practical defensive reference. Apply exercises only to systems, identities, applications, and environments for which you have explicit authorization.

Container security model

Containers reduce some isolation and deployment problems but introduce new layers: images, registries, orchestration APIs, service accounts, secrets, admission controls, and workload identities.

Build-time controls

Use trusted base images, dependency scanning, minimal images, signed artifacts where appropriate, reproducible builds, and clear ownership of images.

Runtime controls

Restrict privileges, avoid unnecessary host access, apply network policies, protect orchestration credentials, monitor unusual workload behavior, and separate sensitive workloads.

Investigation

When a workload behaves unexpectedly, preserve deployment metadata, image identifiers, service-account information, relevant events, network observations, and recent configuration changes before making destructive changes.

Chapter takeaway

The objective is not to memorize individual tools or alerts. Build repeatable reasoning: establish context, define the hypothesis, collect independent evidence, assess business impact, choose the least disruptive effective control, and document what remains uncertain.

Chapter 09: Vulnerability Management & Patch Strategy

This chapter is written as a practical defensive reference. Apply exercises only to systems, identities, applications, and environments for which you have explicit authorization.

Beyond scanning

A vulnerability scanner produces findings, not risk decisions. Effective vulnerability management combines asset criticality, exposure, exploitability, business function, compensating controls, and remediation timelines.

Prioritization

A critical flaw on an isolated test system may be less urgent than a lower-scored weakness on an internet-facing identity service. Use a documented risk model rather than severity labels alone.

Exception management

Every accepted risk should have an owner, reason, compensating control, review date, and expiration condition. Permanent exceptions are usually undocumented technical debt.

Metrics

Track time-to-remediate, overdue risk, exposed critical assets, recurring findings, exception age, and remediation effectiveness.

Chapter takeaway

The objective is not to memorize individual tools or alerts. Build repeatable reasoning: establish context, define the hypothesis, collect independent evidence, assess business impact, choose the least disruptive effective control, and document what remains uncertain.

Chapter 10: Data Security, DLP & Information Classification

This chapter is written as a practical defensive reference. Apply exercises only to systems, identities, applications, and environments for which you have explicit authorization.

Data-first security

Protecting systems without understanding the data they contain makes prioritization difficult. Classify information according to business impact and regulatory or contractual requirements.

Control layers

Use access control, encryption, retention rules, DLP where appropriate, secure sharing, monitoring, and data minimization. Controls should follow the data across storage, processing, and transfer.

DLP tuning

Start with high-confidence sensitive data patterns and meaningful destinations. Excessive alerts train users and analysts to ignore warnings.

Incident questions

What data was involved? Who could access it? Was it actually accessed or only exposed? Was it transferred? Which systems or identities were involved? What evidence supports each conclusion?

Chapter takeaway

The objective is not to memorize individual tools or alerts. Build repeatable reasoning: establish context, define the hypothesis, collect independent evidence, assess business impact, choose the least disruptive effective control, and document what remains uncertain.

Chapter 11: Security Logging & Detection Engineering

This chapter is written as a practical defensive reference. Apply exercises only to systems, identities, applications, and environments for which you have explicit authorization.

Logging architecture

Good logging is an investigation capability. Define which questions analysts must answer, then work backward to the telemetry required. Normalize timestamps, identities, asset identifiers, and event semantics.

Detection quality

A strong detection has a clear hypothesis, known telemetry dependencies, expected false positives, severity criteria, and an investigation procedure. Avoid rules that are impossible to explain.

Testing

Validate detections with benign simulations and controlled test cases. Measure whether telemetry arrived, whether the rule fired, whether enrichment worked, and whether the analyst could understand the alert.

Lifecycle

Hypothesis → telemetry mapping → prototype → validation → tuning → deployment → monitoring → periodic review.

Chapter takeaway

The objective is not to memorize individual tools or alerts. Build repeatable reasoning: establish context, define the hypothesis, collect independent evidence, assess business impact, choose the least disruptive effective control, and document what remains uncertain.

Chapter 12: Threat Modeling for New Projects

This chapter is written as a practical defensive reference. Apply exercises only to systems, identities, applications, and environments for which you have explicit authorization.

Start before deployment

Threat modeling is most valuable while architecture can still change. Identify assets, actors, trust boundaries, sensitive operations, abuse cases, and assumptions.

Practical method

For each component ask what can go wrong, how an attacker could reach it, what the business impact would be, what controls prevent or detect the issue, and who owns the control.

Prioritization

Focus on threats that combine realistic exposure with meaningful impact. Avoid producing a huge list that nobody can maintain.

Deliverable

A useful threat model contains a system diagram, data flows, trust boundaries, top risks, mitigations, residual risk, owners, and review date.

Chapter takeaway

The objective is not to memorize individual tools or alerts. Build repeatable reasoning: establish context, define the hypothesis, collect independent evidence, assess business impact, choose the least disruptive effective control, and document what remains uncertain.

Chapter 13: Third-Party & Supply-Chain Security

This chapter is written as a practical defensive reference. Apply exercises only to systems, identities, applications, and environments for which you have explicit authorization.

Supplier risk

Organizations inherit risk from software vendors, managed services, libraries, integrations, and hardware providers. Security review should therefore continue beyond the organization's own network.

Due diligence

Review security responsibilities, authentication, data handling, breach notification, vulnerability disclosure, update practices, dependency transparency, access requirements, and offboarding.

Software supply chain

Maintain an inventory of critical dependencies and build systems. Restrict who can publish artifacts, protect build credentials, review dependency changes, and verify provenance where practical.

Response

If a supplier incident occurs, identify affected services, credentials, data, versions, trust relationships, and business dependencies before deciding on containment.

Chapter takeaway

The objective is not to memorize individual tools or alerts. Build repeatable reasoning: establish context, define the hypothesis, collect independent evidence, assess business impact, choose the least disruptive effective control, and document what remains uncertain.

Chapter 14: Business Continuity, Backups & Recovery Engineering

This chapter is written as a practical defensive reference. Apply exercises only to systems, identities, applications, and environments for which you have explicit authorization.

Recovery is a security function

Backups help only when they are available, trustworthy, and restorable. Recovery planning should address identity systems, critical applications, configurations, secrets, dependencies, and communication.

Backup principles

Maintain appropriate isolation, access control, retention, integrity checks, and multiple recovery points. Test restoration rather than merely checking that backup jobs completed.

Recovery exercise

Choose a critical service and document dependencies, recovery order, responsible teams, maximum tolerable downtime, data-loss tolerance, validation checks, and rollback conditions.

Security validation

After recovery, verify privileged access, configuration integrity, monitoring, endpoint protection, and the absence of the original incident conditions.

Chapter takeaway

The objective is not to memorize individual tools or alerts. Build repeatable reasoning: establish context, define the hypothesis, collect independent evidence, assess business impact, choose the least disruptive effective control, and document what remains uncertain.

Chapter 15: Security Governance, Metrics & Analyst Career Skills

This chapter is written as a practical defensive reference. Apply exercises only to systems, identities, applications, and environments for which you have explicit authorization.

Governance that works

Security policies should translate into measurable ownership. Every important control needs a responsible team, a verification method, an exception process, and a review cycle.

Useful metrics

Measure risk reduction rather than activity alone: privileged-account coverage, logging coverage, remediation age, tested recovery capability, detection quality, unresolved high-risk exceptions, and incident-response exercise results.

Analyst communication

A strong analyst can explain what happened, what evidence supports it, what remains uncertain, what the business impact is, and what action is recommended.

Continuous improvement

After every significant investigation or exercise, identify one detection improvement, one process improvement, and one preventive control improvement. Record owners and deadlines.

Chapter takeaway

The objective is not to memorize individual tools or alerts. Build repeatable reasoning: establish context, define the hypothesis, collect independent evidence, assess business impact, choose the least disruptive effective control, and document what remains uncertain.

Appendix — New-Volume Quick Reference

Architecture	Trust boundaries → identity → authorization → segmentation → monitoring.
Identity	Account → device → application → action → time → privilege.
Email	Message → authentication → user interaction → campaign scope → identity impact.
Web/API	Asset → endpoint → identity → authorization → data → logging.
Cloud	Principal → control-plane action → resource → configuration → data/workload impact.
Containers	Image → workload → service account → secrets → network → orchestration events.
Vulnerability	Asset criticality → exposure → exploitability → impact → deadline → validation.
Data	Classification → access → movement → exposure → evidence → response.
Detection	Hypothesis → telemetry → rule → validation → tuning → review.
Recovery	Dependency map → restore order → integrity → security validation → business validation.